WAD - Ch. 1-3 Notes

WEB APPLICATION DEVELOPMENT | BE05000281 | Sem 5 | GTU | (Ch 1 - 3)

Compiled BY : **Manthan A. D.** [in]

Ch. 1. INTRO. TO WEB TECHNOLOGIES

1.1 Web Technology

Definition

Web technology refers to the **languages, protocols, software and techniques used to create and access websites and web applications**.

Main technologies

- **HTML** → structure of webpage
- **CSS** → presentation and styling
- **JavaScript** → behavior and interaction
- **Backend technologies** → server-side processing
- **Database** → data storage
- **HTTP** → communication between client and server

Basic web application structure

```
User
  ↓
Browser
  ↓
```

```
Frontend
  ↓
Backend
  ↓
Database
```

Key idea

HTML = Structure

CSS = Style

JavaScript = Behavior

1.2 WWW

WWW = World Wide Web

The World Wide Web is a system of interconnected web pages and resources accessed through the Internet using web browsers and web technologies.

Basic working

```
User
  ↓
Web Browser
  ↓
Internet
  ↓
Web Server
  ↓
Web Page / Data
  ↓
Browser
```

Important terms

Browser: Software used to access websites.

Web Server: Computer/software that receives requests and provides web resources.

URL: Address of a web resource.

HTTP: Protocol used for communication between client and server.

1.3 Evolution of Websites

Websites evolved from simple static pages to interactive web applications.

```
Static Websites
      ↓
Dynamic Websites
      ↓
Interactive Websites
      ↓
Modern Web Applications
```

Static stage

- Fixed content.
- Mainly HTML/CSS.
- Little interaction.

Dynamic stage

- Content can be generated dynamically.
- Server-side processing introduced.
- Database interaction became possible.

Interactive stage

- JavaScript and DOM.
- User interaction.
- APIs and asynchronous communication.

1.4 Client-Server Architecture ⭐⭐⭐⭐⭐

Definition

Client-server architecture is a model in which the **client sends a request**, the **server processes it**, and the **server returns a response**.

Components

Client

Usually a web browser.

Functions:

- Sends request.
- Displays response.
- Handles client-side interaction.

Server

Handles:

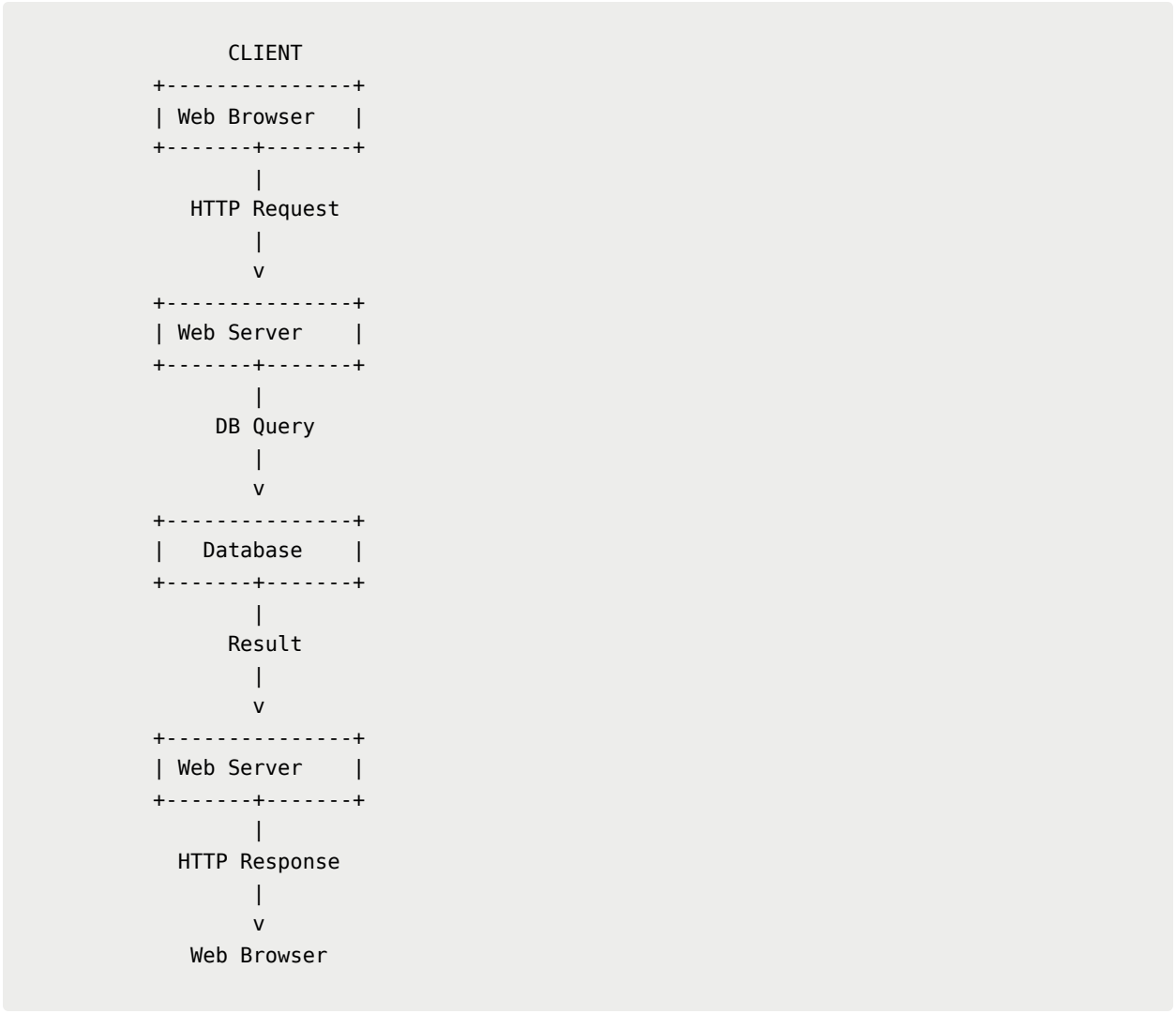
- Business logic.
- Request processing.
- Authentication.
- Database interaction.

Database

Stores:

- User data.
- Transactions.
- Products.
- Other application information.

Diagram



Working

1. User interacts with webpage.
2. Browser creates HTTP request.
3. Request is sent to server.
4. Server processes request.
5. Server may access database.
6. Database returns result.
7. Server sends response.
8. Browser displays result.

Advantages

- Centralized processing.
- Centralized data management.
- Multiple clients can use same server.
- Easier security and maintenance.

1.5 Frontend, Backend and Database

Frontend

Frontend is the **visible part of a web application**.

Main technologies

- HTML
- CSS
- JavaScript

Examples

- Buttons
- Forms
- Navigation
- Page layout
- Animations

Backend

Backend is the **server-side part of the application**.

Responsibilities

- Business logic

- User authentication
- Processing requests
- Database operations
- API handling

Database

A database stores and manages application data.

Examples:

- Student records
- User accounts
- Banking transactions
- Product information

Architecture

```
Frontend
   ↓
Backend
   ↓
Database
```

1.6 Client-Side vs Server-Side Scripting ⭐⭐⭐⭐⭐

This is one of the most repeated topics in your analysis.

Client-Side Scripting

Client-side code executes in the **user's browser**.

Example

JavaScript.

Uses

- Form validation
- DOM manipulation
- Event handling
- User interface interaction

Server-Side Scripting

Server-side code executes on the **web server**.

Uses

- Business logic
- Authentication
- Database operations
- Processing requests

Comparison

Point	Client-Side	Server-Side
Execution	Browser	Server
Main purpose	Interaction	Processing
Common technology	JavaScript	Node.js, PHP, Python etc.
Database access	Usually indirect	Usually direct
Example	Form validation	Login authentication

Memory

```
Client → Browser → User Interaction
Server → Backend → Processing + Database
```

1.7 Static vs Dynamic Websites

⭐⭐⭐⭐⭐

Static Website

A static website displays mostly **fixed content**.

Features

- Fixed HTML content.
- Simple development.
- Usually no database.
- Suitable for basic information pages.

Example

Personal portfolio.

Dynamic Website

A dynamic website generates or changes content according to user requests, application logic or stored data.

Features

- Dynamic content.
- Server-side processing.
- Often database-connected.
- Suitable for interactive applications.

Example

Banking website.

Comparison

Static	Dynamic
Fixed content	Changing/generated content
Simple	More complex
Usually no database	Often uses database
Less interactive	More interactive
Portfolio	Banking/E-commerce

Diagram

```
STATIC

Browser
   |
   ↓
HTML/CSS
   |
   ↓
Fixed Content
```

```
DYNAMIC

Browser
   ↓
Server
   ↓
Application
   ↓
Database
   ↓
Generated Response
```

1.8 Role of Web Technologies in Banking and Finance ⭐⭐⭐

This is a new-syllabus topic specifically identified as a blind spot in the PYQ analysis.

Applications

1. **Online Banking**
   - Check balance.
   - View transactions.
   - Transfer money.
2. **Online Payments**

- Digital payment processing.

3. **Account Management**
   - Update customer information.
4. **Loan and Insurance Services**
   - Online applications and forms.
5. **Trading**
   - View market information and execute transactions.
6. **Customer Support**
   - Online support and service requests.
7. **Authentication**
   - Login and identity verification.

Basic architecture

```
Customer
   ↓
Browser
   ↓
Bank Web Application
   ↓
Bank Server
   ↓
Database
   ↓
Response
```

Important points

- Fast service.
- 24×7 accessibility.
- Reduced manual work.
- Easy access to financial information.
- Security and authentication are important.

Ch. 1 QUICK REVISION

```
WEB TECHNOLOGY
|
├── WWW
├── Browser
├── HTTP
├── Client-Server
├── Frontend
├── Backend
├── Database
├── Client-Side
├── Server-Side
├── Static Website
├── Dynamic Website
└── Banking & Finance
```

Ch. 2. HTML & CSS FUNDAMENTALS

The official syllabus covers HTML document structure, basic tags, tables, forms, media, semantic elements, CSS selectors, colors, backgrounds, fonts, box model, Flexbox, positioning, responsive design, media queries and Bootstrap.

2.1 HTML

HTML = HyperText Markup Language

HTML is used to create the **structure of a webpage**.

Basic document structure

```html
<!DOCTYPE html>

<html>

<head>
    <title>My Page</title>
</head>
```

Structure

```
HTML
|
|
├── HEAD
|   ├── title
|   ├── meta
|   └── CSS links
|
|
|
```

```
<body>

    <h1>Hello World</h1>
    <p>Welcome to WAD</p>

</body>

</html>
```

```
|
└── BODY
    ├── headings
    ├── paragraphs
    ├── images
    ├── tables
    ├── forms
    └── other content
```

2.2 Basic HTML Tags

Tag	Purpose
`<html>`	Root element
`<head>`	Metadata/information
`<title>`	Page title
`<body>`	Visible content
`<h1>` – `<h6>`	Headings
`<p>`	Paragraph
`<a>`	Hyperlink
`<img>`	Image
` `	Line break
`<hr>`	Horizontal line
`<div>`	Generic container
`<span>`	Inline container

Example

```
<h1>Web Application Development</h1>
<p>This is a paragraph.</p>
```

```html
<a href="https://example.com">Visit</a>

<img src="image.jpg" alt="Image">
```

2.3 HTML Lists

Ordered List

Displays numbered items.

```html
<ol>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ol>
```

Unordered List

Displays bullet points.

```html
<ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ul>
```

2.4 HTML Tables ⭐⭐⭐⭐⭐

HTML tables display data in rows and columns.

Important tags

Tag	Meaning
`<table>`	Table
`<tr>`	Row
`<th>`	Heading cell
`<td>`	Data cell

Example

```
<table border="1">

    <tr>
        <th>Name</th>
        <th>Marks</th>
    </tr>

    <tr>
        <td>Manthan</td>
        <td>90</td>
    </tr>

</table>
```

`rowspan`

Used to merge cells **vertically across rows**.

```
<td rowspan="2">Manthan</td>
```

`colspan`

Used to merge cells **horizontally across columns**.

```
<td colspan="2">Student Details</td>
```

Visual

```
            colspan
        <--------------->
+------------------------+
|     Student Details    |
+------------+-----------+
|  Manthan   |   90      |
|            +-----------+
|            |   A       |
+------------+-----------+
   ↑
 rowspan
```

Memory

rowspan = merge rows

colspan = merge columns

2.5 HTML Forms ⭐⭐⭐⭐⭐

Forms are used to **collect user input**.

Basic form

```html
<form>

    Name:
    <input type="text">

    Password:
    <input type="password">

    <input type="submit">

</form>
```

Form Controls

Text Input

```
<input type="text">
```

Password

```
<input type="password" name="pwd">
```

Radio Button

Used when selecting **one option**.

```
<input type="radio" name="gender" value="male"> Male
 <input type="radio" name="gender" value="female"> Female
```

Checkbox

Used for **multiple selections**.

```
<input type="checkbox" name="hobby1" value="reading"> Reading
<input type="checkbox" name="hobby2" value="music"> Music
```

Select

Creates dropdown list.

```
<select>
    <option>Rajkot</option>
    <option>Ahmedabad</option>
</select>
```

Textarea

Used for multi-line input.

```
<textarea></textarea>
```

Submit

```
<input type="submit" value="Submit">
```

2.6 Important Form Attributes - PVR D NCR

Attribute	Purpose
`name`	Identifies field
`value`	Field value
`placeholder`	Hint
`required`	Makes field compulsory
`readonly`	Prevents editing
`disabled`	Disables control
`checked`	Selects radio/checkbox

2.7 HTML Media Elements

Image

```
<img src="photo.jpg" alt="Photo">
```

Audio

```
<audio controls src="song.mp3"></audio>
```

Video

```
<video controls src="video.mp4"></video>
```

Memory

Image → `<img>`

Audio → `<audio>`

Video → `<video>`

2.8 `<div>` and Semantic Elements ⭐⭐⭐⭐

`<div>`

A generic container used to group HTML content.

```
<div class="box">
    Content
</div>
```

It does not describe what the content means.

Semantic Elements

Semantic elements clearly describe their purpose.

Important elements

```
<header>
<nav>
<main>
<section>
<article>
<aside>
<footer>
```

Example

```
<header>
    <h1>My Website</h1>
</header>

<nav>
    Home | About | Contact
</nav>

<main>

    <section>
        <h2>About</h2>
    </section>

    <article>
        <h2>News</h2>
    </article>

</main>

<footer>
    Copyright
</footer>
```

Structure

```
<header>
   ↓
<nav>
   ↓
<main>
 ├── <section>
 └── <article>
   ↓
<footer>
```

Semantic vs `<div>`

Semantic	`<div>`
Meaningful	Generic
Better structure	No semantic meaning
Helps readability	General-purpose container
Better accessibility	Less descriptive

2.9 CSS

CSS = Cascading Style Sheets

CSS is used to **style and format HTML elements**.

Main uses

- Colors
- Fonts
- Backgrounds
- Spacing
- Borders
- Layout
- Positioning
- Responsive design

2.10 Types of CSS ⭐⭐⭐⭐⭐

The PYQ analysis marks this as a high-priority recurring topic.

1. Inline CSS

Written directly inside an HTML element.

```
<p style="color: red;">Hello</p>
```

Advantage

- Quick.

Disadvantage

- Difficult to maintain.

---

2. Internal CSS

Written inside `<style>` in the HTML document.

```
<style>
p {
    color: blue;
}
</style>
```

Advantage

- Useful for a single webpage.

---

3. External CSS

Written in a separate CSS file.

```
<link rel="stylesheet" href="style.css">
```

style.css

```
p {
    color: green;
}
```

Advantage

- Reusable.
- Easy to maintain.
- Suitable for multiple pages.

2.11 CSS Selectors ⭐⭐⭐⭐⭐

Selectors identify which HTML elements should receive CSS styles.

Element Selector

```
p {
    color: red;
}
```

Class Selector

```
.box {
    color: blue;
}
```

ID Selector

```
#title {
    color: green;
}
```

Universal Selector

```
* {
    margin: 0;
```

```
}
```

2.12 Class vs ID ⭐⭐⭐⭐⭐

Class	ID
Starts with `.`	Starts with `#`
Reusable	Intended for unique element
`.box`	`#title`

Example

```
<p class="text">One</p>
<p class="text">Two</p>

<p id="title">Heading</p>
```

2.13 Colors, Backgrounds and Fonts

Color

```
p {
    color: blue;
}
```

Background

```
body {
    background-color: lightgray;
}
```

Font

```css
p {
    font-family: Arial;
    font-size: 20px;
    font-weight: bold;
}
```

Text

```css
p {
    text-align: center;
}
```

2.14 CSS Box Model ⭐⭐⭐⭐

The CSS Box Model consists of:

1. Content
2. Padding
3. Border
4. Margin

Diagram

```
+---------------------------------+
|             MARGIN              |
|   +-------------------------+   |
|   |         BORDER          |   |
|   |   +-----------------+   |   |
|   |   |     PADDING     |   |   |
|   |   |   +----------+  |   |   |
|   |   |   | CONTENT  |  |   |   |
|   |   |   +----------+  |   |   |
|   |   +-----------------+   |   |
|   +-------------------------+   |
+---------------------------------+
```

Meaning

Content: Actual text/image.

Padding: Space between content and border.

Border: Boundary around element.

Margin: Space outside the border.

Example

```
.box {
    width: 200px;
    padding: 20px;
    border: 2px solid black;
    margin: 10px;
}
```

Memory

Padding = Inside

Margin = Outside

2.15 CSS Positioning ⭐⭐⭐⭐

CSS positioning controls where an element appears.

Static

Default position.

```
position: static;
```

Relative

Positioned relative to its normal position.

```
position: relative;
top: 20px;
left: 10px;
```

Absolute

Positioned relative to its nearest positioned ancestor.

```
position: absolute;
top: 10px;
right: 20px;
```

Fixed

Positioned relative to the browser viewport.

```
position: fixed;
```

Sticky

Behaves normally until a specified scrolling position is reached.

```
position: sticky;
```

2.16 Z-Index

`z-index` controls the **stacking order** of overlapping elements.

```
.box1 {
    position: absolute;
    z-index: 2;
}
```

Higher `z-index` generally appears above a lower one.

```
z-index 2 → Front
z-index 1 → Behind
```

2.17 Flexbox ⭐⭐⭐⭐⭐

Flexbox is a CSS layout system for arranging elements in **rows or columns**.

Basic syntax

```css
.container {
    display: flex;
}
```

Important properties

```
flex-direction
justify-content
align-items
flex-wrap
gap
```

Example

```css
.container {
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 20px;
}
```

Meaning

`flex-direction`

Controls row/column direction.

`justify-content`

Aligns items along the main axis.

`align-items`

Aligns items along the cross axis.

`gap`

Controls space between items.

Diagram

```
+----------------------------+
|  [A]   [B]   [C]   [D]     |
+----------------------------+

     ← main axis →
```

2.18 Responsive Web Design ⭐⭐⭐⭐

Responsive web design means a webpage adapts to **different screen sizes**.

Devices

- Desktop
- Laptop
- Tablet
- Mobile

Example

```
Desktop
+----------------------+
| A | B | C | D        |
+----------------------+

Mobile
+------------+
```

```
| A          |
+------------+
| B          |
+------------+
| C          |
+------------+
| D          |
+------------+
```

2.19 Media Queries ⭐⭐⭐⭐

Media queries apply CSS according to screen/device conditions.

Syntax

```css
@media (max-width: 600px) {

    .container {
        flex-direction: column;
    }

}
```

Working

```
Screen Size
    ↓
Media Query
    ↓
Condition satisfied?
    ↓
Apply CSS
```

2.20 Bootstrap ⭐⭐⭐⭐

Bootstrap is a frontend framework that provides predefined styles and responsive layout components.

Features

- Responsive design
- Grid system
- Predefined CSS classes
- Components
- Faster development

Grid system

Bootstrap uses a **12-column grid**.

```
|1|2|3|4|5|6|7|8|9|10|11|12|
```

Example

```
<div class="container">

    <div class="row">

        <div class="col-md-6">
            Left
        </div>

        <div class="col-md-6">
            Right
        </div>

    </div>

</div>
```

Diagram

```
Desktop

|--------- 6 ---------|--------- 6 ---------|
|        Left         |        Right        |

Mobile

|---------------------|
|        Left         |
|---------------------|
|        Right        |
|---------------------|
```

Ch. 2 IMPORTANT CODE PATTERNS

HTML page

```
<!DOCTYPE html>
<html>
<head>
    <title>Page</title>
</head>
<body>
    <h1>Hello</h1>
</body>
</html>
```

External CSS

```
<link rel="stylesheet" href="style.css">
```

Table

```html
<table border="1">
<tr>
    <th>Name</th>
    <th>Marks</th>
</tr>
<tr>
    <td>Manthan</td>
    <td>90</td>
</tr>
</table>
```

Form

```html
<form>
    <input type="text">
    <input type="password">
    <input type="submit">
</form>
```

Flexbox

```css
.container {
    display: flex;
    justify-content: center;
    align-items: center;
}
```

Ch. 3. JAVASCRIPT FUNDAMENTALS

The official syllabus covers variables, datatypes, operators, conditions, loops, functions, arrays, objects, DOM, event handling, local storage, asynchronous programming, callbacks, promises and Fetch API.

3.1 JavaScript

JavaScript is a programming language used to add **logic, dynamic behavior and interactivity** to webpages.

Role of JavaScript

```
HTML
  ↓
Structure

CSS
  ↓
Style

JavaScript
  ↓
Logic + Interaction
```

3.2 Variables

Variables store data values.

`let`

```
let age = 20;
```

Value can be changed.

`const`

```
const pi = 3.14;
```

Used when the variable should not be reassigned.

`var`

```
var name = "Manthan";
```

Older variable declaration method.

Remember

```
let   → can reassign
const → cannot reassign
```

3.3 Data Types

Primitive data types

- String
- Number
- Boolean
- Undefined
- Null
- BigInt
- Symbol

Examples

```
let name = "Manthan";
let age = 20;
let passed = true;
let x;
let y = null;
```

Non-primitive

- Object
- Array

3.4 Operators

Arithmetic

```
+  -  *  /  %  **
```

Comparison

```
==  ===
!=  !==
>   <   >=   <=
```

Logical

```
&&
||
!
```

Assignment

```
=
+=
-=
*=
/=
```

Important

```
==   → loose comparison
===  → strict comparison
```

3.5 Conditional Statements

if

```
if (age >= 18) {
    console.log("Adult");
}
```

if-else

```
if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Minor");
}
```

else-if

```
if (marks >= 90) {
    grade = "A";
}
else if (marks >= 70) {
    grade = "B";
}
else {
    grade = "C";
}
```

3.6 Loops ⭐⭐⭐⭐

for

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

while

```
while (condition) {
    // code
}
```

do-while

```
do {
    // code
} while (condition);
```

Difference

Loop	Use
`for`	Known/repeated count
`while`	Condition-based
`do-while`	Executes at least once

3.7 Functions ⭐⭐⭐

A function is a block of reusable code.

Syntax

```
function add(a, b) {
    return a + b;
}
```

Calling

```
let result = add(2, 3);
```

Parts

```
Function
|
├── Name
├── Parameters
├── Body
└── Return value
```

3.8 Arrays

An array stores multiple values.

```
let fruits = ["Apple", "Banana", "Mango"];
```

Access

```
fruits[0];
```

Common methods

```
push()
pop()
shift()
```

```
unshift()
length
```

Example

```
fruits.push("Orange");
fruits.pop();
```

3.9 Objects

Objects store data in **key-value pairs**.

```
let student = {
    name: "Manthan",
    age: 20,
    branch: "Computer"
};
```

Access

```
student.name;
student["age"];
```

Structure

```
Student
├── name   → Manthan
├── age    → 20
└── branch → Computer
```

3.10 JavaScript Logic Programs

⭐⭐⭐⭐⭐

The PYQ analysis identifies **Prime, Fibonacci, Odd-Even and Pattern Printing** as highly recurring numerical questions.

A. Odd or Even

Logic

```
n % 2 == 0 → Even
otherwise  → Odd
```

Code

```
let n = 10;

if (n % 2 === 0)
    console.log("Even");
else
    console.log("Odd");
```

B. Prime Number

A prime number has exactly two positive factors: **1 and itself**.

Logic

```
Check divisibility from 2 to √n
If divisible → Not Prime
Otherwise → Prime
```

Code

```
let n = 17;
let prime = true;

if (n <= 1) {
    prime = false;
}

for (let i = 2; i <= Math.sqrt(n); i++) {

    if (n % i === 0) {
        prime = false;
        break;
    }

}

if (prime)
    console.log("Prime");
else
    console.log("Not Prime");
```

C. Fibonacci Series

Sequence:

```
0 1 1 2 3 5 8 13 ...
```

Logic

```
Next = Previous1 + Previous2
```

Code

```
let n = 10;
let a = 0;
let b = 1;

for (let i = 0; i < n; i++) {

    console.log(a);

    let c = a + b;
    a = b;
    b = c;

}
```

D. Pattern Printing

Output

```
*
**
***
****
*****
```

Code

```
let n = 5;

for (let i = 1; i <= n; i++) {

    let row = "";

    for (let j = 1; j <= i; j++) {
        row += "*";
```

```
    }

    console.log(row);
}
```

Main concept

Pattern printing = Nested loops

3.11 DOM ⭐⭐⭐⭐⭐

DOM = Document Object Model

DOM represents an HTML document as a **tree of objects**.

Diagram

```
Document
   |
  HTML
 /    \
Head   Body
        |
       Div
      /   \
     H1   Button
```

JavaScript can access and modify this tree.

3.12 DOM Selection Methods

`getElementById()`

```
document.getElementById("title");
```

`querySelector()`

```
document.querySelector(".box");
```

`querySelectorAll()`

```
document.querySelectorAll("p");
```

3.13 DOM Manipulation ⭐⭐⭐⭐⭐

Change content

```
document.getElementById("title").innerHTML = "Hello";
```

Change style

```
document.getElementById("title").style.color = "red";
```

General flow

```
HTML Page
    ↓
DOM Tree
    ↓
JavaScript selects element
    ↓
Modify element
    ↓
Updated webpage
```

3.14 Event Handling ⭐⭐⭐⭐⭐

An event is an action that occurs in the webpage.

Common events

- `click`
- `mouseover`
- `mouseout`
- `keydown`
- `keyup`
- `submit`
- `change`
- `load`

Event listener

```
button.addEventListener("click", function() {
    alert("Button Clicked");
});
```

Flow

```
User Action
    ↓
Event
    ↓
Event Listener
    ↓
Function
    ↓
Action
```

3.15 Popup Boxes

Alert

Displays a message.

```
alert("Hello");
```

Confirm

Asks for confirmation.

```
let result = confirm("Are you sure?");
```

Returns:

- `true` → OK
- `false` → Cancel

Prompt

Accepts input.

```
let name = prompt("Enter name");
```

Memory

```
alert   → Tell
confirm → Ask
prompt  → Take input
```

3.16 Regular Expression Validation

⭐⭐⭐⭐⭐

Regex is used to match a required input pattern.

Common patterns

Username

```
^[A-Za-z0-9_]{3,15}$
```

Email

```
^[^\s@]+@[^\s@]+\.[^\s@]+$
```

Mobile

```
^[0-9]{10}$
```

Password

```
^(?=.*[A-Za-z])(?=.*\d).{8,}$
```

JavaScript

```
let regex = /^[0-9]{10}$/;

if (regex.test(mobile)) {
    console.log("Valid");
} else {
    console.log("Invalid");
}
```

Flow

```
User Input
    ↓
Regex Pattern
    ↓
Match?
  ┌──┴──┐
Yes   No
```

```
 ↓     ↓
Valid Error
```

3.17 Local Storage ⭐⭐⭐⭐

Local Storage stores data in the browser using **key-value pairs**.

Store

```
localStorage.setItem("name", "Manthan");
```

Read

```
let name = localStorage.getItem("name");
```

Remove one

```
localStorage.removeItem("name");
```

Remove all

```
localStorage.clear();
```

Diagram

```
JavaScript
    ↓
Local Storage
+-------------------+
| key       value   |
| name      Manthan |
| theme     dark    |
+-------------------+
```

Methods

Method	Purpose
`setItem()`	Store
`getItem()`	Read
`removeItem()`	Remove one
`clear()`	Remove all

3.18 Synchronous Programming ⭐⭐⭐⭐

In synchronous programming, statements execute **one after another**.

```
Task 1
  ↓
Task 2
  ↓
Task 3
```

If one task takes time, later tasks wait.

3.19 Asynchronous Programming ⭐⭐⭐⭐

In asynchronous programming, a long-running operation can complete later while other code continues executing.

```
Task 1 ————————→
Task 2 ———→
Task 3 ——→
        ↓
     Result
```

Example

```
console.log("Start");

setTimeout(() => {
    console.log("Later");
}, 2000);

console.log("End");
```

Output

```
Start
End
Later
```

3.20 Synchronous vs Asynchronous

Synchronous	Asynchronous
Sequential	Non-blocking style
Waits for previous operation	Other work can continue
Simple execution	Useful for long operations
Can block	Helps responsiveness

3.21 Callback ⭐⭐⭐⭐

A callback is a **function passed as an argument to another function** and executed later or when required.

Example

```
function greet(name, callback) {

    console.log("Hello " + name);
```

```
    callback();
}

greet("Manthan", function() {
    console.log("Welcome");
});
```

Diagram

```
Function A
    ↓
Receives callback
    ↓
Callback Function
    ↓
Execute
```

Common uses

- Event handling
- Timers
- Asynchronous operations

3.22 Promise ⭐⭐⭐⭐

A Promise represents the eventual result of an asynchronous operation.

States

```
          Pending
         /       \
        ↓         ↓
   Fulfilled   Rejected
```

Example

```
let promise = new Promise((resolve, reject) => {

    let success = true;

    if (success)
        resolve("Operation Successful");
    else
        reject("Operation Failed");

});
```

Handling

```
promise
    .then(result => console.log(result))
    .catch(error => console.log(error));
```

Memory

```
.then()  → success
.catch() → failure
```

3.23 Fetch API ⭐⭐⭐⭐

Fetch API is used to make HTTP requests from JavaScript.

Basic syntax

```
fetch(url)
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log(error));
```

Working

1. `fetch()` sends request.
2. Fetch returns a Promise.
3. Server sends response.
4. Response is converted to JSON.
5. Data is processed.
6. Errors are handled by `catch()`.

Diagram

```
JavaScript
    ↓
fetch()
    ↓
HTTP Request
    ↓
Server / API
    ↓
HTTP Response
    ↓
JSON
    ↓
JavaScript
```

3.24 Fetch with `async/await`

```
async function getData() {

    try {

        let response = await fetch(url);

        let data = await response.json();

        console.log(data);
```

```
    } catch (error) {

        console.log(error);

    }
}
```

Memory

```
fetch()
  ↓
Promise
  ↓
response
  ↓
response.json()
  ↓
data
```

3.25 Complete DOM + Event + Fetch Connection

This is the important conceptual connection between the advanced topics.

```
User
 ↓
Clicks Button
 ↓
JavaScript Event
 ↓
Event Handler
 ↓
Fetch API
 ↓
Server / API
 ↓
Response
 ↓
Promise
```

```
 ↓
JSON Data
 ↓
DOM Manipulation
 ↓
Updated Webpage
```

This is essentially how a modern interactive webpage works.

3.26 IMPORTANT DIFFERENCES

Client-Side vs Server-Side

Client-Side	Server-Side
Browser	Server
UI/interaction	Processing
JavaScript	Node.js/PHP/Python
Example: validation	Example: authentication

Static vs Dynamic

Static	Dynamic
Fixed	Generated
Simple	Complex
Usually no DB	Often DB
Portfolio	Banking

Inline vs Internal vs External CSS

Type	Location
Inline	Inside element
Internal	`<style>` in HTML
External	Separate `.css` file

Class vs ID

Class	ID
`.`	`#`
Reusable	Unique
`.box`	`#title`

Margin vs Padding

```
Margin  → Outside border
Padding → Inside border
```

Rowspan vs Colspan

```
rowspan → Rows
colspan → Columns
```

Radio vs Checkbox

```
Radio    → One choice
Checkbox → Multiple choices
```

Alert vs Confirm vs Prompt

```
Alert   → Message
Confirm → Confirmation
Prompt  → Input
```

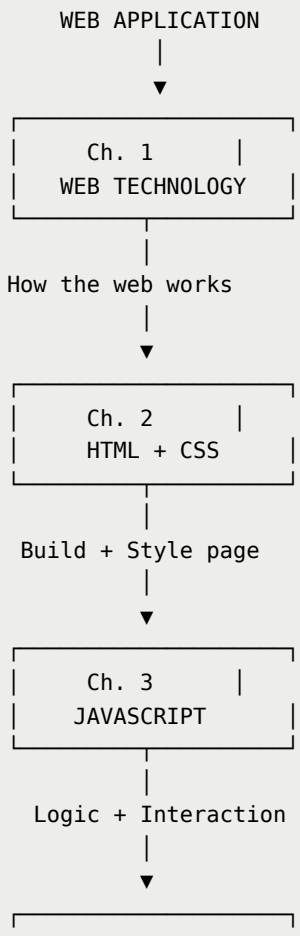
Synchronous vs Asynchronous

```
Sync  → Wait
Async → Continue
```

Callback vs Promise

```
Callback → Function passed to another function
Promise  → Represents future async result
```

COMPLETE Ch. CONNECTION



```
                  | DOM + Events       |
                  | Local Storage      |
                  | Async + Callback   |
                  | Promise + Fetch    |
                  └────────────────────┘
```

LAST-MINUTE REVISION ORDER

Since you are short on time, revise in this order:

First Priority

1. **Client-Server Architecture**
2. **Client-Side vs Server-Side**
3. **Static vs Dynamic**
4. **HTML Structure + Basic Tags**
5. **HTML Forms**
6. **HTML Tables + rowspan/colspan**
7. **CSS Types**
8. **Class vs ID**
9. **CSS Box Model**
10. **Flexbox**
11. **DOM**
12. **Event Handling**
13. **Regex Validation**
14. **Prime / Fibonacci / Odd-Even / Pattern**

Second Priority

1. CSS Positioning + Z-index
2. Responsive Design + Media Queries

3. Bootstrap Grid
4. Semantic HTML
5. Local Storage
6. Synchronous vs Asynchronous
7. Callbacks
8. Promises
9. Fetch API

New-syllabus topics that should not be skipped

The PYQ analysis explicitly flags **Banking & Finance, Box Model, Semantic HTML, Responsive Design/Media Queries, Fetch/Promises, Local Storage and Static vs Dynamic** as areas with limited or no old-PYQ history.

10-MINUTE MEMORY SHEET

```
HTML  = Structure
CSS   = Style
JS    = Behavior

CLIENT = Browser
SERVER = Processing
DB     = Storage

STATIC = Fixed
DYNAMIC = Generated

TABLE:
<tr> <th> <td>
rowspan = row
colspan = column

FORM:
text
password
radio
checkbox
select
textarea
submit
```

```
RESPONSIVE:
@media


JS:
let
const
datatype
operator
if
loop
function
array
object


DOM:
getElementById
querySelector
innerHTML
style

EVENT:
click
mouseover
```

```
CSS:
inline
internal
external

SELECTORS:
element
.class
#id

BOX:
content
padding
border
margin

LAYOUT:
Flexbox
justify-content
align-items

POSITION:
static
relative
absolute
fixed
sticky
z-index
```

```
keydown
submit

VALIDATION:
Regex


STORAGE:
setItem
getItem
removeItem
clear

ASYNC:
callback
promise
fetch

PROMISE:
pending
fulfilled
rejected

FETCH:
request
response
JSON
data
```

Final exam concept chain

Ch. 1: Understand the web and client-server system.

Ch. 2: Build and style the webpage.

Ch. 3: Add logic, interaction, storage and API communication.

> **Compiled BY** : Manthan A. D. [in]